

Experiment 8

Arman Imam

EEE3342C

25Fall 0011

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically, and Artix-7 FPGA Board

Objective:

The objective of experiment 8 is to introduce the user how to implement Finite State Machines. For part 1, the user is asked to create a Moore and Mealy Machine based on the figures in the experiment brief, and to test it with a specific input sequence. For part 2, the user is asked to make a Bit Sequence Detector with Mealy Machine from PISO Shift Register, and to test it with a specific input sequence, and on an FPGA board (which is optional). For part 3, the user is asked to create a 4-bit multiple and divider using Moore FSM; Though this part is optional.

Design Steps:

I began by creating a single project file, where I created the specified source files for part 1 and 2. For part 1, I created the Moore and Mealy Machine based on the figures in the design brief, and used the input sequence '0 1 0 1 1 0 1 0 1 0 0 1' for the test bench, and took a screenshot of the waveform. For part 2, I created a Mealy Machine, where the output is only 1 when the sequence "1101" is detected. I wrote a module with the 8-bit PISO Shift Register, and a module that combines the mealy machine and shift register, which accepts an 8 bit input. I chose not to do part 3, which is optional.

Part 1:

Verilog RTL of Moore machine.

The Moore machine has 3 inputs (clk, reset, in) and 1 output (out). The code for the Moore and Mealy machine are roughly the same, except for the last few lines of code. The output for the Moore machine is only dependent on the current state. The output is only 1, if the current state is A or B.

```

1 | `timescale 1ns / 1ps
2 | //MOORE MACHINE
3 | module Part1_1(
4 |     //clk: clock, reset: async\active high, in: input to decide next state
5 |     input wire clk, reset, in,
6 |     output reg out //moore-style output
7 | );
8 |     //State Encoding
9 |     localparam STATE_A = 2'b00;
10 |    localparam STATE_B = 2'b01;
11 |    localparam STATE_C = 2'b10;
12 |    localparam STATE_D = 2'b11;
13 |
14 |    //State Registers:
15 |    reg [1:0] current_state, next_state;
16 |
17 |    //Next STATE Combinationl Logic
18 |    always @(*) begin
19 |        case (current_state)
20 |            STATE_A: begin
21 |                if (in) next_state = STATE_B;
22 |                else next_state = STATE_A;
23 |            end
24 |            STATE_B: begin
25 |                if (!in) next_state = STATE_B;
26 |                else next_state = STATE_C;
27 |            end
28 |            STATE_C: begin
29 |                if (in) next_state = STATE_D;
30 |                else next_state = STATE_A;
31 |            end
32 |            STATE_D: begin
33 |                if (in) next_state = STATE_A;
34 |                else next_state = STATE_B;
35 |            end
36 |            default: next_state = STATE_A;
37 |        endcase
38 |    end
39 |
40 |    //State UPDATE
41 |    always @(posedge clk or posedge reset) begin
42 |        if (reset) current_state <= STATE_A;
43 |        else current_state <= next_state;
44 |    end
45 |    //EVERYTHING ABOVE WILL BE THE SAME FOR MOORE AND MEALY
46 |    //Output Logic
47 |    //MOORE: Output depends only on current state
48 |    always @(*) begin
49 |        case (current_state)
50 |            STATE_A: out = 1'b1;
51 |            STATE_B: out = 1'b1;
52 |            STATE_C: out = 1'b0;
53 |            STATE_D: out = 1'b0;
54 |            default: out = 1'b0;
55 |        endcase
56 |    end
57 |
58 | endmodule

```

Verilog RTL of Mealy machine

The Mealy machine has 3 inputs (clk, reset, in) and 1 output (out). The code for the Moore and Mealy machine are roughly the same, except for the last few lines of code. The output for the Mealy machine is dependent on the current state AND the input. For example, The output is 1 when current state is STATE_B, and input is 0.

```

1  `timescale 1ns / 1ps
2  //MEALY MACHINE
3  module Part1_2(
4      //clk: clock, reset: async\active high, in: input to decide next state
5      input wire clk, reset, in,
6      output reg out //mealy-style output
7  );
8      //State Encoding
9      localparam STATE_A = 2'b00;
10     localparam STATE_B = 2'b01;
11     localparam STATE_C = 2'b10;
12     localparam STATE_D = 2'b11;
13
14     //State Registers:
15     reg [1:0] current_state, next_state;
16
17     //Next STATE Combinationl Logic
18     always @(*) begin
19         case (current_state)
20             STATE_A: begin
21                 if (in) next_state = STATE_B;
22                 else next_state = STATE_A;
23             end
24             STATE_B: begin
25                 if (!in) next_state = STATE_B;
26                 else next_state = STATE_C;
27             end
28             STATE_C: begin
29                 if (in) next_state = STATE_D;
30                 else next_state = STATE_A;
31             end
32             STATE_D: begin
33                 if (in) next_state = STATE_A;
34                 else next_state = STATE_B;
35             end
36             default: next_state = STATE_A;
37         endcase
38     end
39
40     //State UPDATE
41     always @(posedge clk or posedge reset) begin
42         if (reset) current_state <= STATE_A;
43         else current_state <= next_state;
44     end
45     //EVERYTHING ABOVE WILL BE THE SAME FOR MOORE AND MEALY
46     //Output Logic
47     //MEALY: Output depends on both current state AND input
48     always @(*) begin
49         case (current_state)
50             STATE_A: begin
51                 out = 1'b1;
52             end
53             STATE_B: begin
54                 if (in) out = 1'b0;
55                 else out = 1'b1;
56             end
57             STATE_C: begin
58                 if (in) out = 1'b0;
59                 else out = 1'b1;
60             end
61             STATE_D: begin
62                 out = 1'b1;
63             end
64             default: out = 1'b0;
65         endcase
66     end
67
68 endmodule

```

Input Value	Current State	Next State	Mealy Output	Moore Output
0	A	A	1	1
0	B	B	1	1
0	C	A	1	0
0	D	B	1	0
1	A	B	1	1
1	B	C	0	1
1	C	D	0	0
1	D	A	1	0

Create testbench using this input sequence below with a 10ns delay after each delivered bit: 0 1 0 1 1 0 1 0 1 0 0 1

Moore: This test bench tests the input sequence through each digit after 10 nanoseconds

```

1  `timescale 1ns / 1ps
2
3  module part1_1_sim();
4      reg clk, reset, in;
5      wire out;
6
7      Part1_1 PDA (.clk(clk), .reset(reset), .in(in), .out(out));
8      always #5 clk = ~clk;
9      initial begin
10         clk = 0;
11         reset = 1;
12         in = 0; #20;
13         reset = 0; #10;
14
15         in = 0; #10;
16         in = 1; #10;
17         in = 0; #10;
18         in = 1; #10;
19         in = 1; #10;
20         in = 0; #10;
21         in = 1; #10;
22         in = 0; #10;
23         in = 1; #10;
24         in = 0; #10;
25         in = 0; #10;
26         in = 1; #10;
27         #10;
28         $finish;
29     end
30 endmodule

```

Mealy: This test bench tests the input sequence through each digit after 10 nanoseconds

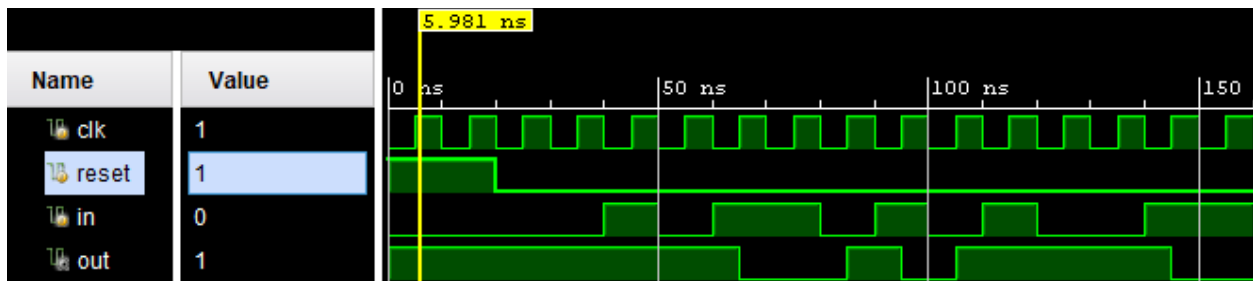
```

1  timescale 1ns / 1ps
2
3  module part1_2_sim();
4      reg clk, reset, in;
5      wire out;
6
7      Part1_2 PDB (.clk(clk),.reset(reset), .in(in), .out(out));
8      always #5 clk = ~clk;
9      initial begin
10         clk = 0;
11         reset = 1;
12         in = 0; #20;
13         reset = 0; #10;
14
15         in = 0; #10;
16         in = 1; #10;
17         in = 0; #10;
18         in = 1; #10;
19         in = 1; #10;
20         in = 0; #10;
21         in = 1; #10;
22         in = 0; #10;
23         in = 1; #10;
24         in = 0; #10;
25         in = 0; #10;
26         in = 1; #10;
27         #10;
28         $finish;
29     end
30 endmodule

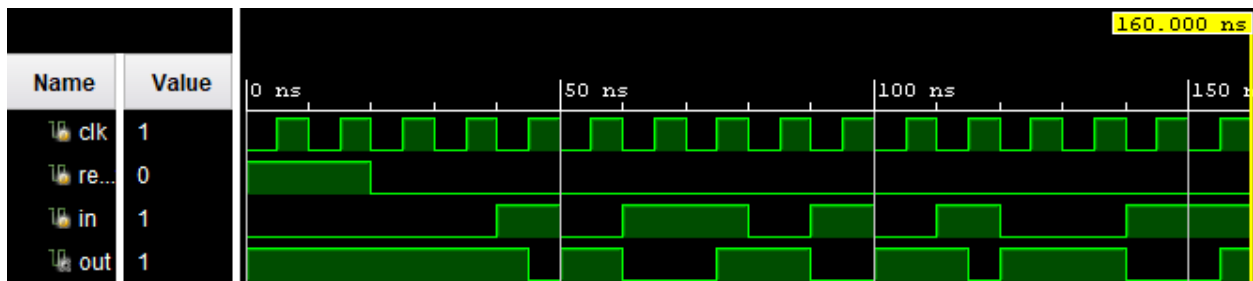
```

Screenshot of Waveform of Moore and Mealy machine output.

Moore: The output is only 1, if the current state is A or B. When it reaches state C or D, then the output is 0.



Mealy: The output is 1, when current state is A or D, and when input is 1 for state B and C.



Part 2:

Verilog RTL of Mealy machine.

There are 3 inputs(clk, reset, in), and 1 output(out). The output is only 1, when the current state is D, and the input is 1 at that point. This code specifically outputs 1 when the sequence is '1101', so the state has to go from A to B (1), B to C (1), C to C (0), and C to D with input 1.

```
1  `timescale 1ns / 1ps
2  //Bit Sequence Detector with Mealy Machine from PISO Register
3  module part2_1(
4      //clk: clock, reset: async\active high, in: input to decide next state
5      input wire clk, reset, in,
6      output reg out //mealy-style output
7  );
8      //State Encoding
9      localparam STATE_A = 2'b00;
10     localparam STATE_B = 2'b01;
11     localparam STATE_C = 2'b10;
12     localparam STATE_D = 2'b11;
13
14     //State Registers:
15     reg [1:0] current_state, next_state;
16
17     //Next STATE Combinationl Logic
18     always @(*) begin
19         case (current_state)
20             STATE_A: begin
21                 if (in) next_state = STATE_B;
22                 else    next_state = STATE_A;
23             end
24             STATE_B: begin
25                 if (!in) next_state = STATE_A;
26                 else    next_state = STATE_C;
27             end
28             STATE_C: begin
29                 if (in) next_state = STATE_C;
30                 else    next_state = STATE_D;
31             end
32             STATE_D: begin
33                 if (in) next_state = STATE_B;
34                 else    next_state = STATE_A;
35             end
36             default: next_state = STATE_A;
37         endcase
38     end
```

```

39 |
40 | //State UPDATE
41 | always @(posedge clk or posedge reset) begin
42 |     if (reset) current_state <= STATE_A;
43 |     else      current_state <= next_state;
44 | end
45 | //EVERYTHING ABOVE WILL BE THE SAME FOR MOORE AND MEALY
46 | //Output Logic
47 | //MEALY: Output depends on both current state AND input
48 | always @(*) begin
49 |     case (current_state)
50 |         STATE_A: out = 1'b0;
51 |         STATE_B: out = 1'b0;
52 |         STATE_C: out = 1'b0;
53 |         STATE_D: out = in ? 1'b1 : 1'b0; // Only here!
54 |         default: out = 1'b0;
55 |     endcase
56 | end
57 |
58 | endmodule

```

Verilog for 8bit PISO: 8 bit shift register

```

1 | `timescale 1ns / 1ps
2 | //PISO Shift Register 8-bit
3 | //8 bit shift register
4 | module part2_2(
5 |     input [7:0] in,
6 |     input clk, en, load,
7 |     output reg out
8 | );
9 |     reg q[7:0];
10 |
11 |     always @(posedge clk)begin
12 |         //LOAD stage
13 |         if (load == 1) begin
14 |             q[0] <= in[0];
15 |             q[1] <= in[1];
16 |             q[2] <= in[2];
17 |             q[3] <= in[3];
18 |             q[4] <= in[4];
19 |             q[5] <= in[5];
20 |             q[6] <= in[6];
21 |             q[7] <= in[7];
22 |         end
23 |         //Shift Stage
24 |         if (en == 1) begin
25 |             q[0] <= 0;
26 |             q[1] <= q[0];
27 |             q[2] <= q[1];
28 |             q[3] <= q[2];
29 |             q[4] <= q[3];
30 |             q[5] <= q[4];
31 |             q[6] <= q[5];
32 |             q[7] <= q[6];
33 |             out <= q[7];
34 |         end
35 |     end
36 | endmodule

```

Combining both modules:

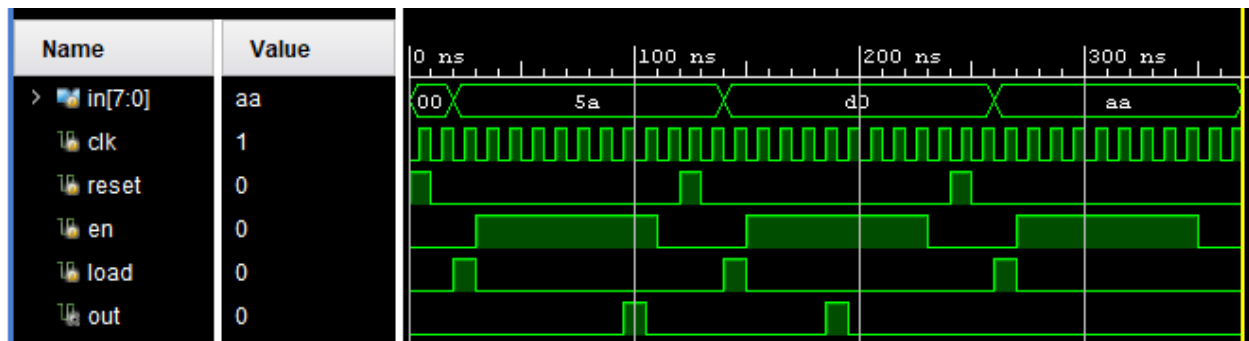
```

1  `timescale 1ns / 1ps
2  //Combining Mealy and PISO Shift register
3  //Output of mealy, and input of PISO
4  //out = 1, when value shows '1101'
5  module part2_3(
6      input [7:0] in,
7      input clk, en, load, reset,
8      output wire out
9  );
10     wire bit;
11
12     //PISO Shift Register
13     part2_2 ABT(.in(in), .clk(clk), .en(en), .load(load), .out(bit));
14     //Mealy
15     part2_1 ACT(.in(bit), .clk(clk), .reset(reset), .out(out));
16 endmodule
17

```

Screenshot of waveform of Mealy machine working with PISO shift register using the asynchronous reset.

There are two instances where the sequence “1101” is detected, hence when output is 1. The first instance was near the beginning of the first test, and the second instance was in the beginning of Test 2.



Create testbench using this input sequence from PISO below with a 10ns delay after each delivered bit:

o Test 1: 0 1 0 1 1 0 1 0

o Test 2: 1 1 0 1 0 0 0 0 Reset 1 0 1 0 1 0 1 0

```

1  `timescale 1ns / 1ps
2
3  module part2_sim();
4      reg [7:0] in;
5      reg clk, reset, en, load;
6      wire out;
7
8      part2_3 uut (
9          .in(in),
10         .clk(clk),
11         .reset(reset),
12         .en(en),
13         .load(load),
14         .out(out)
15     );
16
17     // 10ns clock period
18     always #5 clk = ~clk;
19
20     initial begin
21         // Initialize
22         clk = 0;
23         reset = 0;
24         en = 0;
25         load = 0;
26         in = 8'b0;
27
28         // TEST 1: 0 1 0 1 1 0 1 0
29         reset = 1;
30         #10;
31         reset = 0;
32         #10;
33
34         in = 8'b01011010;
35         load = 1;
36         #10;
37         load = 0;
38
39         en = 1;
40         #80; // 8 bits x 10ns
41
42         en = 0;
43
44         // TEST 2: 1 1 0 1 0 0 0 0
45         #10;
46         reset = 1;
47         #10;
48         reset = 0;
49         #10;
50
51         in = 8'b11010000;
52         load = 1;
53         #10;
54         load = 0;
55
56         en = 1;
57         #80;
58         en = 0;
59
60         // RESET
61         #10;
62         reset = 1;
63         #10;
64         reset = 0;
65         #10;
66
67         // Continue TEST 2: 1 0 1 0 1 0 1 0
68         in = 8'b10101010;
69         load = 1;
70         #10;
71         load = 0;
72
73         en = 1;
74         #80;
75         en = 0;
76
77         #20;
78         $finish;
79     end
80 endmodule

```

Q&A:

What is the difference between a Moore and Mealy machine? Explain the pros and cons.

Moore Machine outputs depend on current states, while Mealy Machine outputs depend on current states and inputs.

What size would your state registers need to be if you had 16 states?

4 bits, because $2^4 = 16$.

Where do you set the outputs of a Mealy machine?

On transitions, based on current state and input.

Where do you set the outputs of a Moore machine?

Inside the states, based only on the state.

In your own words, do you think FSMs can replace other digital circuits?

Explain your reasoning.

Only in cases where logic needs to be controlled in a systematic way. It would not be ideal for computational tasks, or data processing.

Test Plan:

Part 1:

Moore:

```
1  `timescale 1ns / 1ps
2
3  module part1_1_sim();
4      reg clk, reset, in;
5      wire out;
6
7      Part1_1 PDA (.clk(clk),.reset(reset), .in(in), .out(out));
8      always #5 clk = ~clk;
9      initial begin
10         clk = 0;
11         reset = 1;
12         in = 0; #20;
13         reset = 0; #10;
14
15         in = 0; #10;
16         in = 1; #10;
17         in = 0; #10;
18         in = 1; #10;
19         in = 1; #10;
20         in = 0; #10;
21         in = 1; #10;
22         in = 0; #10;
23         in = 1; #10;
24         in = 0; #10;
25         in = 0; #10;
26         in = 1; #10;
27         #10;
28         $finish;
29     end
30 endmodule
```

Mealy:

```

1  timescale 1ns / 1ps
2
3  module part1_2_sim();
4      reg clk, reset, in;
5      wire out;
6
7      Part1_2 PDB (.clk(clk),.reset(reset), .in(in), .out(out));
8      always #5 clk = ~clk;
9      initial begin
10         clk = 0;
11         reset = 1;
12         in = 0; #20;
13         reset = 0; #10;
14
15         in = 0; #10;
16         in = 1; #10;
17         in = 0; #10;
18         in = 1; #10;
19         in = 1; #10;
20         in = 0; #10;
21         in = 1; #10;
22         in = 0; #10;
23         in = 1; #10;
24         in = 0; #10;
25         in = 0; #10;
26         in = 1; #10;
27         #10;
28         $finish;
29     end
30 endmodule

```

Part 2:

```

1  `timescale 1ns / 1ps
2
3  module part2_sim();
4      reg [7:0] in;
5      reg clk, reset, en, load;
6      wire out;
7
8      part2_3 uut (
9          .in(in),
10         .clk(clk),
11         .reset(reset),
12         .en(en),
13         .load(load),
14         .out(out)
15     );
16
17     // 10ns clock period
18     always #5 clk = ~clk;
19
20     initial begin
21         // Initialize
22         clk = 0;
23         reset = 0;
24         en = 0;
25         load = 0;
26         in = 8'b0;
27
28         // TEST 1: 0 1 0 1 1 0 1 0
29         reset = 1;
30         #10;
31         reset = 0;
32         #10;
33
34         in = 8'b01011010;
35         load = 1;
36         #10;
37         load = 0;
38
39         en = 1;
40         #80; // 8 bits x 10ns
41
42         en = 0;
43
44         // TEST 2: 1 1 0 1 0 0 0 0
45         #10;
46         reset = 1;
47         #10;
48         reset = 0;
49         #10;
50
51         in = 8'b11010000;
52         load = 1;
53         #10;
54         load = 0;
55
56         en = 1;
57         #80;
58         en = 0;
59
60         // RESET
61         #10;
62         reset = 1;
63         #10;
64         reset = 0;
65         #10;
66
67         // Continue TEST 2: 1 0 1 0 1 0 1 0
68         in = 8'b10101010;
69         load = 1;
70         #10;
71         load = 0;
72
73         en = 1;
74         #80;
75         en = 0;
76
77         #20;
78         $finish;
79     end
80 endmodule

```

Part 3:

Conclusion

This experiment was a success. The results were as expected for all parts of the experiment. However, I initially had trouble understanding the logic of the Finite State Machine, but I did understand it eventually. Implementing Finite State Machines has furthered my understanding of how its made, and how it can be used in real world applications.